Self-Paced Module: Java Threads and Concurrency Tools

Java Programming · Grade 12 TVL · Cielito Zamora Senior High School · Week 16, November 3–6, 2026 · Continues the File Handling module

Name: _____
Section: _____ Date: _____

Day	Time	Topic	What you will do
Mon, Nov 2	—	No class — All Souls' Day	Nothing to hand in. Read "Where we left off" below if you like.
Tue, Nov 3	2 hours	What a thread is; its life cycle; extending `Thread`	Pages 1–2, Steps 1–3. Answer the **Tuesday strip** (page 7, middle).
Wed, Nov 4	2 hours	`Runnable`; waiting for others with `CountDownLatch`	Page 3, Steps 4–5. Answer the **Wednesday strip** (page 7, top).
Thu, Nov 5	3 hours	`CyclicBarrier` and `Semaphore`	Page 4, Steps 6–7. Answer the **Thursday strip** (page 8, bottom). Start your Performance Task.
Fri, Nov 6	3 hours	**PERFORMANCE TASK**	Page 6. Finish, present, and answer questions. Answer the **Friday strip** (page 8, top).

How to use this module: Work in order. Type every sample code and run it — do not just read it. Each day ends with a **LIPLET**, a short written quiz on **page 7 or 8**. Tear off **only** that day's strip, so you keep the rest of the module until Friday. The bottom of page 7 is a practice box for you to keep (not handed in).

No internet needed. Everything you need is in this module and on your computer with Java (JDK) installed. Keep every class in its own file, all in one folder. Compile with `javac *.java`, run with `java Main`. Stuck on an error? See the **Error Guide on page 5**.

WHERE WE LEFT OFF — your Vehicle program from the File Handling module

Your folder should already have: `Vehicle.java`, `Jeepney.java`, `Tricycle.java`, `Bus.java`, `OverloadException.java`, `InvalidDistanceException.java`, and `VehicleNotFoundException.java`. None of them change this week. You will replace `Main.java` many times, so save a copy first as `Main_week15.txt`. This week you also add five small new classes, one per Step that needs it. The text files from last week are not used this week.

The problem this week fixes: every program you wrote so far does **one thing at a time**. If the program waits for a slow job, such as loading passengers, **everything else waits too**. A real terminal has many things going on at once. Java can do this with **threads**.

PART A — WHAT IS A THREAD?

Definition: A **process** is a program that is running, with its own memory. A **thread** is one line of work **inside** a process. Every Java program starts with one thread, called `main`. Your program can start more threads, and they run **at the same time** and **share** the memory of the process. Threads of the same process can see the same objects; two different processes cannot.

ANALOGY The **terminal** is the process: it has one building, one logbook, and one set of vehicles. The **dispatchers** are the threads. With one dispatcher (`main`), every job waits for the one before it. With three dispatchers, one can load passengers, one can collect fares, and one can call the next jeepney, all in the same terminal, all looking at the same logbook. The risk: two dispatchers writing in the same line of the logbook at the same time. So this week, **each thread uses only its own data**.

	Process	Thread
What it is	a whole program that is running	one line of work inside a process
Memory	has its own; other processes cannot see it	shares the process's memory with its sibling threads
In the terminal	the whole terminal and its logbook	one dispatcher working in that terminal
In Java	`java Main` starts one	`main` is the first; `start()` makes more

Day	Java tool	Its job
Tue	`Thread`, `start()`, `join()`	make a new line of work; start it; wait until it is finished
Wed	`Runnable`, `CountDownLatch`	describe a job apart from the worker; wait until a number of jobs have reported
Thu	`CyclicBarrier`, `Semaphore`	make a group wait for each other again and again; let only a few use something at once

The five new classes you will write this week

Class	When	Its job
`DispatchThread`	Tue, Step 1	a `Thread` that collects the fares of one vehicle over several trips
`BoardingTask`	Wed, Step 4	a `Runnable` job that boards groups of passengers
`ReadyReport`	Wed, Step 5	a `Runnable` that reports "ready" to a `CountDownLatch`
`ConvoyLeg`	Thu, Step 6	a `Runnable` that drives to two stops and waits for the convoy at each
`LoadingBay`	Thu, Step 7	a `Runnable` that takes one of two bays, loads, then gives the bay back

Before you start: open a terminal in your folder and run `javac *.java`. If it prints nothing, your Vehicle files are fine.

Words to know this week. Concurrent: several things in progress at the same time. **Start**: ask Java to begin a new thread. **Join**: wait until another thread is finished. **Sleep**: pause the thread for some milliseconds (1000 ms = 1 second). **Interrupt**: a request to a sleeping or waiting thread to stop; it shows up as `InterruptedException`, a **checked** exception from Week 13.

Timing note: the outputs in this module come from short pauses (0.1 to 0.4 seconds) set on purpose, so the order is the same every run. On a very slow computer a line or two could appear in a different order.

TUESDAY STEP 1 — A Thread is a worker; run() is its job

Definition: To make your own worker, write a class that **extends Thread** and **override** `run()`: the code the new thread will do. You never call `run()` yourself. You call `start()`, and Java creates the new line of work and calls `run()` in it. `join()` makes the calling thread wait until the other thread is finished.

STEP 1 Make a new file `DispatchThread.java` (left), then replace `Main.java` (right). The thread's job: collect 5 trips of fares, 0.1 second per trip.

```
public class DispatchThread extends Thread {      // a Thread that does dispatch work
    private Vehicle vehicle;
    private int trips;
    private double collected = 0;
    private String ranOn = "not run yet";

    public DispatchThread(Vehicle vehicle, int trips) {
        super(vehicle.getPlateNumber());   // the thread's name is the plate
        this.vehicle = vehicle;
        this.trips = trips;
    }

    @Override
    public void run() {                    // the job this thread does
        ranOn = Thread.currentThread().getName();
        for (int i = 1; i <= trips; i++) {
            collected = collected + vehicle.quoteFare(5);
            try {
                Thread.sleep(100);         // one trip takes time (0.1 second)
            } catch (InterruptedException e) {
                return;                    // stop early if interrupted
            }
        }
    }

    public double getCollected() { return collected; }
    public String getRanOn() { return ranOn; }
}
```

```
public class Main {
    public static void main(String[] args) {
        System.out.println("main() runs on thread: " + Thread.currentThread().getName());
        Vehicle jeep = new Jeepney("NXA 123", "Mang Ramon", "Monumento-Divisoria");
        DispatchThread t = new DispatchThread(jeep, 5);

        System.out.println("Before start(): " + t.getState());
        t.start();                         // a NEW thread begins run()
        try {
            t.join();                      // main waits here until t is finished
        } catch (InterruptedException e) {
            System.out.println("main was interrupted");
        }
        System.out.println("After join(): " + t.getState());
        System.out.println(t.getName() + " ran on thread: " + t.getRanOn());
        System.out.println("Collected after 5 trips: P" + t.getCollected());
    }
}
```

Output:

```
main() runs on thread: main
Before start(): NEW
After join(): TERMINATED
NXA 123 ran on thread: NXA 123
Collected after 5 trips: P75.0
```

Look closely: the thread is `NEW` before `start()` and `TERMINATED` after its `run()` ended. The thread was named by the plate number (`super(...)` passes the name up to `Thread`, just as `Jeepney`'s constructor passes values up to `Vehicle` in Week 12). 5 trips of P15.00 = P75.0.

Check yourself: (1) Change the 5 in `new DispatchThread(jeep, 5)` to 10 and run. Predict the collected amount first. (P150.0, because each trip adds P15.00; the Output text still says "5 trips" — fix that string.) (2) Why must `t.join()` be inside `try` and `catch (InterruptedException e)`? (`join()` and `sleep()` can throw this **checked** exception, as in Week 13.)

Step 1 in order — who is doing what?

#	Thread `main`	Thread `NXA 123`
1	prints `main() runs on thread: main`; creates the `DispatchThread` (state **NEW**)	does not exist as a running thread yet
2	prints `Before start(): NEW`, then calls `start()`	is born and begins `run()` (**RUNNABLE**)
3	calls `join()` and **waits** (state **WAITING**)	5 trips; each ends with `sleep(100)` (**TIMED_WAITING**)
4	wakes up when `run()` ends; prints the state, the thread name, the total	**TERMINATED**

Predict before you run: what will the first line say? Write your guess here: _____ (The program's own thread is named **main**.)

The life of a thread — its states

Thread state	What it means	When you see it
`NEW`	The Thread object exists; `start()` was not called yet.	Step 1, before `start()`
`RUNNABLE`	Started: running, or waiting its turn on the processor.	just after `start()`
`TIMED_WAITING`	Paused for a set time, e.g. inside `Thread.sleep(100)`.	each trip in Step 1
`WAITING`	Paused with no time limit, e.g. `join()` or `await()` with no wait time.	main, inside `join()`
`TERMINATED`	The `run()` method has finished. It cannot be started again.	Step 1, after `join()`

TUESDAY STEP 2 — The big mistake: calling run() instead of start()

STEP 2 Replace `Main.java`. The first worker is started the **wrong** way, the second the **right** way.

```
public class Main {
    public static void main(String[] args) {
        Vehicle trike = new Tricycle("TRC 456", "Kuya Ben", "Bagong Silang TODA");
        Vehicle bus = new Bus("BUS 789", "Aling Rosa", 50);

        DispatchThread wrong = new DispatchThread(trike, 2);
        wrong.run();                        // WRONG: just a normal method call
        System.out.println(wrong.getName() + " ran on thread: " + wrong.getRanOn());

        DispatchThread right = new DispatchThread(bus, 2);
        right.start();                      // RIGHT: starts a new thread
        try {
            right.join();
        } catch (InterruptedException e) {
            System.out.println("main was interrupted");
        }
        System.out.println(right.getName() + " ran on thread: " + right.getRanOn());
    }
}
```

Output:

```
TRC 456 ran on thread: main
BUS 789 ran on thread: BUS 789
```

Read it carefully: the tricycle "ran on thread: **main**". `wrong.run()` is only an ordinary method call: the main thread did the whole job itself and nothing ran at the same time. The bus ran on a thread named **BUS 789**, a real new thread.

start() and run() side by side

	`wrong.run()`	`right.start()`
New thread made?	No	Yes
Who does the job?	the caller, `main`	the new thread (named by the plate)
Can `main` do other work meanwhile?	No: it is busy until `run()` returns	Yes, until it calls `join()`
Step 2 output	`TRC 456 ran on thread: main`	`BUS 789 ran on thread: BUS 789`

Predict before Step 3: three workers, 5 trips of 0.1 second each. If they ran one after another, they would take about _____ seconds. Started together, about _____ seconds. (1.5 and 0.5.)

TUESDAY STEP 3 — Three threads at the same time

STEP 3 Replace `Main.java`. Start all three **first**, then join all three. The results are printed **after** the joins, so they always come out in the same order.

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<DispatchThread> crew = new ArrayList<>();
        crew.add(new DispatchThread(new Jeepney("NXA 123", "Mang Ramon", "Monumento-Divisoria"), 5));
        crew.add(new DispatchThread(new Tricycle("TRC 456", "Kuya Ben", "Bagong Silang TODA"), 5));
        crew.add(new DispatchThread(new Bus("BUS 789", "Aling Rosa", 50), 5));

        long begin = System.currentTimeMillis();
        for (DispatchThread t : crew) {
            t.start();                      // all three run AT THE SAME TIME
        }
        for (DispatchThread t : crew) {
            try {
                t.join();                   // wait for each one to finish
            } catch (InterruptedException e) {
                System.out.println("main was interrupted");
            }
        }
        long ms = System.currentTimeMillis() - begin;

        double total = 0;
        for (DispatchThread t : crew) {     // results in a FIXED order, after join
            System.out.println(t.getName() + ": P" + t.getCollected());
            total = total + t.getCollected();
        }
        System.out.println("Total collected: P" + total);
        System.out.println("Finished in under 1 second? " + (ms < 1000));
    }
}
```

Output:

```
NXA 123: P75.0
TRC 456: P350.0
BUS 789: P137.5
Total collected: P562.5
Finished in under 1 second? true
```

Look closely: each worker needs 5 × 0.1 s = 0.5 s. Done one after another, the three would take about 1.5 s. Started together they finish in about 0.5 s, so `under 1 second? true`.

Check yourself: in `DispatchThread`, change `sleep(100)` to `sleep(300)` and run. Predict the last line. (**false**: each worker now needs 1.5 s, even though all three still run together.) Change it back.

Try this, then undo it: delete the second `for` loop of Step 3 (the one with `join()`) and run. The totals print **too early**, before the threads finish. On our computer: NXA 123: P15.0, TRC 456: P70.0, BUS 789: P27.5, Total P112.5, which is one trip each. Your numbers may differ, but they will be far below P75.0, P350.0 and P137.5. That is why Rule 2 says: `join()` before you read results.

Four rules for threads this week

#	Rule	Why
1	Call **start()**, never `run()`, to begin a thread.	Only `start()` makes a new line of work (Step 2).
2	**join()** every thread before you read its results.	Without it, `main` may read the answer before the thread has finished.
3	Each thread changes only **its own** data.	Two threads changing one variable at the same time can give wrong answers. You will learn the fix in a later lesson.
4	Always handle `InterruptedException`.	`sleep`, `join`, `await` and `acquire` all throw it (checked exception, Week 13).

Step 3 as a timeline (each trip lasts 0.1 second)

Time	NXA 123 (jeepney)	TRC 456 (tricycle)	BUS 789 (bus)
0.0 s	`main` calls `start()` three times; the three threads begin together		
0.1 – 0.4 s	trips 1 to 4	trips 1 to 4	trips 1 to 4
0.5 s	trip 5, then **TERMINATED**	trip 5, then **TERMINATED**	trip 5, then **TERMINATED**
about 0.5 s	`main` was waiting in `join()`; now it prints the three totals and adds them		

Check your folder after Step 3. Classes: `Main.java`, the 7 class files from last week, and the new `DispatchThread.java`. Keep every file. Tomorrow you add `BoardingTask.java` and `ReadyReport.java`.

LIPLET · TUESDAY Stop here and answer the **TUESDAY strip (middle of page 7)**, in pen. Tear it off and submit it.

WEDNESDAY PART B — Runnable: the job, apart from the worker

Definition: `Runnable` is an **interface**: a promise that a class has certain methods. This one has just one, `void run()`. A class that **implements Runnable** describes **a job**. To get it done, hand the job to a `Thread`: `new Thread(job, "name")`. This is the usual way, because the job class can still **extend another class** if it needs to, but a class can extend only one parent, and `Thread` would use it up.

ANALOGY `extends Thread` is like hiring a person **who comes with one job fixed to them** (a loader who can only ever load). `Runnable` is writing the job on a **work order** ("load these groups of passengers") and giving the work order to any available worker. The same order can be given to any worker; the worker is the `Thread`, the order is the `Runnable`.

	extends Thread	implements Runnable
You write	a worker class with `run()`	a job class with `run()`, then `new Thread(job)`
To start it	`worker.start()`	`new Thread(job).start()`
Can the class extend another class?	No (it already extends Thread)	Yes

STEP 4 Make `BoardingTask.java` (below), then replace `Main.java`. It uses Week 13's `board()` and `OverloadException`: each job catches its **own** exception.

```
public class BoardingTask implements Runnable {   // a JOB, not a thread
    private Vehicle vehicle;
    private int[] groups;                  // passengers arriving, group by group
    private String result = "not run yet";

    public BoardingTask(Vehicle vehicle, int[] groups) {
        this.vehicle = vehicle;
        this.groups = groups;
    }

    @Override
    public void run() {
        try {
            for (int g : groups) {
                vehicle.board(g);          // Week 13: may throw OverloadException
                Thread.sleep(50);
            }
            result = "all boarded";
        } catch (OverloadException e) {
            result = "stopped: " + e.getMessage();
        } catch (InterruptedException e) {
            result = "interrupted";
        }
    }

    public String getResult() { return result; }
}
```

```
public class Main {
    public static void main(String[] args) {
        Vehicle jeep = new Jeepney("NXA 123", "Mang Ramon", "Monumento-Divisoria");
        Vehicle trike = new Tricycle("TRC 456", "Kuya Ben", "Bagong Silang TODA");

        BoardingTask jeepJob = new BoardingTask(jeep, new int[] {5, 8, 6});
        BoardingTask trikeJob = new BoardingTask(trike, new int[] {2, 2, 1});

        Thread t1 = new Thread(jeepJob, "Jeep loader");  // job -> Thread
        Thread t2 = new Thread(trikeJob, "Trike loader");
        t1.start();
        t2.start();
        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.out.println("main was interrupted");
        }
        System.out.println(t1.getName() + ": " + jeepJob.getResult());
        System.out.println(t2.getName() + ": " + trikeJob.getResult());
        jeep.showInfo();
        trike.showInfo();
    }
}
```

Output of Step 4:

```
Jeep loader: all boarded
Trike loader: stopped: TRC 456 is full! Capacity 4, but 5 tried to ride.
NXA 123 | Driver: Mang Ramon | Passengers: 19/20
TRC 456 | Driver: Kuya Ben | Passengers: 4/4
```

Look closely: the trike's job stopped when 5 people tried to ride a 4-seat tricycle (Week 13's `OverloadException`), but the jeepney's job was not affected. Each job caught its own exception inside `run()`. A job that catches its own problems cannot hurt the others.

Check yourself: change the jeepney's groups to `{5, 8, 8}` and run. Predict the first line. (`Jeep loader: stopped: NXA 123 is full! Capacity 20, but 21 tried to ride.` and the jeepney then shows 13/20, because the last group was refused.) Change it back.

WEDNESDAY PART C — CountDownLatch: wait until everyone has reported

Definition: A `CountDownLatch` starts with a number, such as 4. Each worker calls `countDown()` when it is done, which lowers the number by 1. A thread that calls `await()` **waits until the number is 0**, then goes on. A latch is **one-shot**: once at 0, it cannot be reset.

ANALOGY The dispatcher says, "We leave when **4** vehicles report ready." Each driver radios in when ready. The dispatcher keeps a tally: 4, 3, 2, 1, 0. At 0 she blows the whistle. It does not matter **who** reports first — only that all 4 have.

STEP 5 Make `ReadyReport.java` (left), then replace `Main.java` (right). The vehicles take different times to get ready: 400, 100, 300, and 200 milliseconds.

```
import java.util.concurrent.CountDownLatch;

public class ReadyReport implements Runnable {
    private String plate;
    private int delayMs;                   // ms to get ready
    private CountDownLatch latch;

    public ReadyReport(String plate, int delayMs, CountDownLatch latch) {
        this.plate = plate;
        this.delayMs = delayMs;
        this.latch = latch;
    }

    @Override
    public void run() {
        try {
            Thread.sleep(delayMs);
            System.out.println(plate + " reports: ready");
        } catch (InterruptedException e) {
            System.out.println(plate + " was interrupted");
        } finally {
            latch.countDown();             // count goes down by 1, no matter what
        }
    }
}
```

```
import java.util.concurrent.CountDownLatch;

public class Main {
    public static void main(String[] args) {
        CountDownLatch latch = new CountDownLatch(4);      // wait for 4 reports
        new Thread(new ReadyReport("NXA 123", 400, latch)).start();
        new Thread(new ReadyReport("TRC 456", 100, latch)).start();
        new Thread(new ReadyReport("BUS 789", 300, latch)).start();
        new Thread(new ReadyReport("NYB 321", 200, latch)).start();

        System.out.println("Dispatcher waits. Reports still needed: " + latch.getCount());
        try {
            latch.await();                 // main stops here until the count is 0
        } catch (InterruptedException e) {
            System.out.println("Dispatcher was interrupted");
        }
        System.out.println("Reports still needed: " + latch.getCount());
        System.out.println("All vehicles ready. Dispatch begins!");
    }
}
```

Output:

```
Dispatcher waits. Reports still needed: 4
TRC 456 reports: ready
NYB 321 reports: ready
BUS 789 reports: ready
NXA 123 reports: ready
Reports still needed: 0
All vehicles ready. Dispatch begins!
```

Read it carefully: the reports come in order of **speed** (TRC 100 ms first, NXA 400 ms last), not the order the threads were started. `countDown()` sits in `finally` (Week 13) so the count goes down even if something goes wrong in the job. Without it, `await()` could wait forever.

Check yourself: change `new CountDownLatch(4)` to `(5)` and run. Only 4 jobs call `countDown()`, so the count stops at 1 and the program **never ends**. Press **Ctrl+C** to stop it, then change it back.

Time	What happens	Latch count
0 ms	The dispatcher creates `CountDownLatch(4)`, starts 4 threads, and calls `await()`: `main` waits	4
100 ms	TRC 456 reports, then `countDown()`	3
200 / 300 ms	NYB 321 reports; then BUS 789 reports	2, then 1
400 ms	NXA 123 reports; the count reaches 0, so `await()` returns	**0** → go!

Check your folder after Step 5. Classes: `Main.java`, the 7 class files from last week, `DispatchThread.java`, `BoardingTask.java`, and `ReadyReport.java`. Tomorrow you add `ConvoyLeg.java` and `LoadingBay.java`.

LIPLET · WEDNESDAY Stop here and answer the **WEDNESDAY strip (top of page 7)**, in pen. Tear it off and submit it.

THURSDAY PART D — CyclicBarrier: wait for each other, again and again

Definition: A `CyclicBarrier(3)` makes a group of **3** threads wait at a meeting point. Each calls `await()`. When the **3rd** arrives, all three are released together, and the barrier is **ready to use again** (that is the "cyclic"). `await()` gives back a number: **0 for the last thread to arrive**. Difference from a latch: here the **workers wait for each other**, and the barrier can be reused.

ANALOGY Three jeepneys travel as a **convoy**. At every stop they wait for each other, and leave only when all three have arrived. At the next stop they wait for each other again. The same "rule of the convoy" is used at every stop — one barrier, reused.

STEP 6 Make `ConvoyLeg.java` (left), then replace `Main.java` (right). It also needs `BrokenBarrierException`, a checked exception that `await()` can throw.

```java
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;

public class ConvoyLeg implements Runnable {
    private String plate;
    private int delayMs;
    private CyclicBarrier barrier;

    public ConvoyLeg(String plate, int delayMs, CyclicBarrier barrier) {
        this.plate = plate;
        this.delayMs = delayMs;
        this.barrier = barrier;
    }

    // drive to a stop, then wait there until the whole convoy has arrived
    private void driveTo(String stop) throws InterruptedException, BrokenBarrierException {
        Thread.sleep(delayMs);
        System.out.println(plate + " arrived at " + stop);
        int stillComing = barrier.await(); // wait for the others
        if (stillComing == 0) {            // 0 = the LAST one to arrive
            System.out.println("-- All here at " + stop + ". The convoy moves on together!");
        }
    }

    @Override
    public void run() {
        try {
            driveTo("Monumento");
            driveTo("Recto");              // the SAME barrier is used again
        } catch (InterruptedException | BrokenBarrierException e) {
            System.out.println(plate + " left the convoy: " + e);
        }
    }
}
```

```java
import java.util.concurrent.CyclicBarrier;

public class Main {
    public static void main(String[] args) {
        CyclicBarrier barrier = new CyclicBarrier(3);  // 3 in the convoy
        Thread a = new Thread(new ConvoyLeg("NXA 123", 300, barrier));
        Thread b = new Thread(new ConvoyLeg("NYB 321", 100, barrier));
        Thread c = new Thread(new ConvoyLeg("BUS 789", 200, barrier));
        a.start();
        b.start();
        c.start();
        try {
            a.join();
            b.join();
            c.join();
        } catch (InterruptedException e) {
            System.out.println("main was interrupted");
        }
        System.out.println("Convoy finished.");
    }
}
```

Output:

```
NYB 321 arrived at Monumento
BUS 789 arrived at Monumento
NXA 123 arrived at Monumento
-- All here at Monumento. The convoy moves on together!
NYB 321 arrived at Recto
BUS 789 arrived at Recto
NXA 123 arrived at Recto
-- All here at Recto. The convoy moves on together!
Convoy finished.
```

Look closely: in both stops, NYB 321 arrives first and NXA 123 last, and the "All here" line comes after the last arrival. **Check yourself:** change `CyclicBarrier(3)` to `(4)`. Only 3 vehicles exist, so all three wait at Monumento **forever** (Ctrl+C to stop). Change it back.

THURSDAY PART E — Semaphore: only a few at a time

Definition: A `Semaphore(2)` holds **2 permits**. `acquire()` takes one permit, or **waits** if none is left. `release()` gives one back. So at most 2 threads are in the protected part at once. `availablePermits()` tells how many are free. Put `release()` in `finally` so a permit is never lost.

ANALOGY The terminal has **2 loading bays**. A vehicle that arrives takes a bay if one is free; otherwise it waits in line. When a vehicle leaves, it frees its bay for the next one. The permits are the bays.

	CountDownLatch	CyclicBarrier	Semaphore
Picture it as	a tally to reach 0	a meeting point for a group	a fixed number of bays
Who waits?	the thread that calls `await()`	every thread that arrives	whoever finds no free permit
Use again?	No, one-shot	Yes, automatically	Yes, acquire and release

STEP 7 Make `LoadingBay.java` (left), then replace `Main.java` (right). Four vehicles arrive 100 ms apart, but there are only 2 bays and loading takes 400 ms.

```java
import java.util.concurrent.Semaphore;

public class LoadingBay implements Runnable {
    private String plate;
    private int arriveMs;                  // when this vehicle reaches the terminal
    private Semaphore bays;

    public LoadingBay(String plate, int arriveMs, Semaphore bays) {
        this.plate = plate;
        this.arriveMs = arriveMs;
        this.bays = bays;
    }

    @Override
    public void run() {
        try {
            Thread.sleep(arriveMs);
            System.out.println(plate + " arrives. Free bays: " + bays.availablePermits());
            bays.acquire();                // take a bay, or WAIT until one is free
            try {
                System.out.println("   " + plate + " is loading in a bay");
                Thread.sleep(400);         // loading takes time
                System.out.println("   " + plate + " leaves the bay");
            } finally {
                bays.release();            // give the bay back, always
            }
        } catch (InterruptedException e) {
            System.out.println(plate + " was interrupted");
        }
    }
}
```

```java
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Semaphore;

public class Main {
    public static void main(String[] args) {
        Semaphore bays = new Semaphore(2);                 // 2 bays only
        List<Thread> line = new ArrayList<>();
        line.add(new Thread(new LoadingBay("NXA 123", 0, bays)));
        line.add(new Thread(new LoadingBay("TRC 456", 100, bays)));
        line.add(new Thread(new LoadingBay("BUS 789", 200, bays)));
        line.add(new Thread(new LoadingBay("NYB 321", 300, bays)));
        for (Thread t : line) {
            t.start();
        }
        for (Thread t : line) {
            try {
                t.join();
            } catch (InterruptedException e) {
                System.out.println("main was interrupted");
            }
        }
        System.out.println("All loaded. Free bays: " + bays.availablePermits());
    }
}
```

Output:

```
NXA 123 arrives. Free bays: 2
   NXA 123 is loading in a bay
TRC 456 arrives. Free bays: 1
   TRC 456 is loading in a bay
BUS 789 arrives. Free bays: 0
NYB 321 arrives. Free bays: 0
   NXA 123 leaves the bay
   BUS 789 is loading in a bay
   TRC 456 leaves the bay
   NYB 321 is loading in a bay
   BUS 789 leaves the bay
   NYB 321 leaves the bay
All loaded. Free bays: 2
```

Read it carefully: BUS 789 and NYB 321 arrived when **Free bays: 0**, so they waited. When NXA 123 left, BUS 789 took its bay. When TRC 456 left, NYB 321 took that one. At no time were more than 2 vehicles loading.

Check yourself: change `Semaphore(2)` to `(1)`. How many vehicles load at once now? (One. The others wait in line, and the last line still says `Free bays: 1`.)

Step 7 as a timeline (2 bays; loading takes 400 ms)

Time	Bay 1	Bay 2	Waiting in line
0 / 100 ms	NXA 123 loads (0 ms)	TRC 456 loads (100 ms)	—
200–300 ms	NXA 123	TRC 456	BUS 789, then NYB 321 (no free permit)
400 ms	**BUS 789** takes the bay NXA left	TRC 456	NYB 321
500 ms	BUS 789	**NYB 321** takes the bay TRC left	—

Check your folder after Step 7. Last week's 7 class files, `Main.java`, and 5 new classes: `DispatchThread`, `BoardingTask`, `ReadyReport`, `ConvoyLeg`, `LoadingBay`. Save `Main.java` as `Main_week16.txt`.

LIPLET · THURSDAY Stop here and answer the **THURSDAY strip (bottom of page 8)**, in pen. Tear it off, submit it, then start page 6.

ERROR GUIDE — no internet? Look up your error here

If you see…	It means…	Fix it by…
`unreported exception InterruptedException; must be caught or declared to be thrown`	`Thread.sleep()`, `join()`, `await()` or `acquire()` was used outside a try.	Put it inside `try { } catch (InterruptedException e) { }`.
`cannot find symbol … symbol: class CountDownLatch`	A missing import.	Add `import java.util.concurrent.CountDownLatch;` (or `CyclicBarrier`, `Semaphore`).
`no suitable constructor found for Thread(Jeepney) … Jeepney cannot be converted to Runnable`	You gave `new Thread(...)` an object whose class does not implement `Runnable`.	Pass a job from a class that says `implements Runnable`.
`Exception in thread "main" java.lang.IllegalThreadStateException`	The same Thread object had `start()` called twice. A thread can run only once.	Make a **new** Thread object for each run.
`ran on thread: main` when you expected the worker's name	You called `run()` instead of `start()`. Nothing ran at the same time.	Call `start()` (Step 2).
`Exception in thread "Bad trip" InvalidDistanceException: …`, and the program carries on	An **unchecked** exception inside one thread stops only that thread. `main` keeps going.	Catch the exception inside `run()`, like Step 4.
The lines stop, but the program **never ends**	An `await()` is waiting for a count or a group that never fills: e.g. `CountDownLatch(5)` but only 4 `countDown()` calls.	Press **Ctrl+C**. Match the number to the real count of threads; use `finally` for `countDown()`.

HOW THIS WEEK CONNECTS TO EARLIER WEEKS

Week	What you learned then	Where it came back this week
11–12	classes and objects; inheritance; overriding	every helper is an object; `DispatchThread extends Thread` overrides `run()` (Step 1)
13	checked exceptions; `try`, `catch`, `finally`	`InterruptedException` everywhere; `OverloadException` caught inside a job (Step 4); `countDown()` and `release()` in `finally` (Steps 5, 7)
14	`List` and `ArrayList`	`List<DispatchThread>` and `List<Thread>` keep many threads to start and join (Steps 3, 7)
15	files and try-with-resources	the same habit of cleaning up: a permit or a count must always be given back

WHICH TOOL FOR WHICH JOB?

The job in the terminal	Use
Do a job on its own line of work, at the same time as `main`	`Thread` (extend it) or `Runnable` (give it to a `Thread`)
Wait until **one** thread has finished	`join()`
Wait **once** until 4 vehicles have all reported ready	`CountDownLatch(4)`
3 vehicles must **all arrive** before any moves on, at stop after stop	`CyclicBarrier(3)`
Only 2 vehicles may use the loading bays at the same time	`Semaphore(2)`

METHODS YOU USED THIS WEEK

Method	What it does
`t.start()`	begins a new thread, which then runs `run()`
`t.join()`	waits until thread `t` is finished
`Thread.sleep(100)`	pauses the current thread for 100 milliseconds
`Thread.currentThread().getName()`	the name of the thread that is running this line
`t.getState()`	NEW, RUNNABLE, TIMED_WAITING, WAITING or TERMINATED
`latch.countDown()` / `latch.await()`	lower the count by 1 / wait until the count is 0
`latch.getCount()`	how many reports are still needed
`barrier.await()`	wait for the whole group; gives 0 to the last one to arrive
`bays.acquire()` / `bays.release()`	take a permit (or wait) / give it back
`bays.availablePermits()`	how many permits are free right now

SAY IT IN YOUR OWN WORDS — a self-test (no one checks this)

Cover the module and write what each word means. Then open the module and check.

Word	In my own words
process	
thread	
start() / join()	
Runnable	
CountDownLatch	
CyclicBarrier	
Semaphore	

SPOT THE BUG — write what happens, then check the Error Guide above

```
Thread t = new Thread(job);
t.start();
t.start();
```

A: What happens?

```
public static void main(String[] a) {
    Thread.sleep(500);
}
```

B: What happens?

```
// 3 threads started, but:
CountDownLatch latch =
    new CountDownLatch(4);
```

C: What happens?

CHALLENGES for fast finishers (optional)

1. **Step 3:** add a fourth worker, `NYB 321` (Mang Tonyo's jeepney on `Sangandaan-Recto`), with 5 trips. Predict the new total first. (P637.5.)
2. **Step 5:** change the four delays so the reports come in the order NXA 123, BUS 789, NYB 321, TRC 456. Which delay does each vehicle need? (NXA 100, BUS 200, NYB 300, TRC 400.)
3. **Step 7:** make loading take 200 ms instead of 400 ms and run. Does any vehicle still wait? Explain using the arrival times.

Looking ahead to your Performance Task (page 6). Each helper in *your* program becomes a thread. A `CountDownLatch` can mean "the store opens when all helpers are ready." A `Semaphore` can mean "only 2 customers at the counter." Start thinking about which helpers your theme has.

THU-FRI PART F — PERFORMANCE TASK: MAKE YOUR PROGRAM DO MANY THINGS AT ONCE

Open your own program from your Four Pillars Performance Task (your sari-sari store, pets, gadgets, school personnel, or shapes theme). Give it **helpers that work at the same time**, and make them wait for each other the right way. Start on Thursday after the Thursday strip; finish and present on Friday.

Requirements checklist — tick each box when done:

- ☐ At least **3 helper objects** from your theme, each doing its own job on its own thread
- ☐ Helpers built with `extends Thread` or `implements Runnable`; each has a **name** that is printed
- ☐ Threads begin with `start()` (never `run()`)
- ☐ Each thread changes only **its own data**, no shared variable
- ☐ A `CountDownLatch`, with `countDown()` in `finally`, and `await()` in `main`
- ☐ A second tool: `Semaphore` **or** `CyclicBarrier`, with a story that fits your theme
- ☐ `Thread.sleep()` shows work takes time; every `InterruptedException` is handled
- ☐ Results are printed **after** everyone has finished, in a fixed order
- ☐ Bad data in one helper is caught inside its `run()`; the others still finish
- ☐ A comment on each part, e.g. `// LATCH: store opens when both cashiers finish`

Reuse this week's code. Your helper class works like `BoardingTask` (Step 4) or `DispatchThread` (Step 1). The latch works like Step 5; the barrier like Step 6; the semaphore like Step 7. The example below shows only the latch: **you must add a second tool.**

Day	Time plan (two 3-hour days)
Thursday first half	Steps 6 and 7 on page 4; Thursday strip
Thursday second half	Fill in the plan on this page; write your helper class and start your threads. *Before you leave: two or more helpers run together and print their names.*
Friday first half	Latch, second tool, bad-data handling; a classmate tries to break it
Friday second half	Presentations and Q&A. *Friday strip handed in.*

Example for one theme (sari-sari store: two cashiers). Do yours your own way. `Cashier.java`:

```
import java.util.concurrent.CountDownLatch;

public class Cashier implements Runnable {
    private String name;
    private String[] items;
    private int[] prices;
    private int startDelay;
    private CountDownLatch done;
    private int total = 0;

    public Cashier(String name, String[] items, int[] prices,
                   int startDelay, CountDownLatch done) {
        this.name = name;
        this.items = items;
        this.prices = prices;
        this.startDelay = startDelay;
        this.done = done;
    }

    @Override
    public void run() {
        try {
            Thread.sleep(startDelay);
            for (int i = 0; i < items.length; i++) {
                total += prices[i];
                System.out.println(name + " rang up " + items[i] + " P" + prices[i]);
                Thread.sleep(200);
            }
        } catch (InterruptedException e) {
            System.out.println(name + " was interrupted");
        } finally {
            done.countDown();
        }
    }

    public int getTotal() { return total; }
}
```

`Main.java`:

```
import java.util.concurrent.CountDownLatch;

public class Main {
    public static void main(String[] args) {
        CountDownLatch done = new CountDownLatch(2);
        Cashier ana = new Cashier("Ana", new String[] {"Bread", "Rice"},
                new int[] {12, 55}, 0, done);
        Cashier ben = new Cashier("Ben", new String[] {"Soap", "Coffee"},
                new int[] {18, 9}, 100, done);
        new Thread(ana, "Ana's counter").start();
        new Thread(ben, "Ben's counter").start();
        try {
            done.await();
        } catch (InterruptedException e) {
            System.out.println("Closing early");
        }
        System.out.println("Ana's total: P" + ana.getTotal());
        System.out.println("Ben's total: P" + ben.getTotal());
        System.out.println("Store total: P" + (ana.getTotal() + ben.getTotal()));
    }
}
```

Output:

```
Ana rang up Bread P12
Ben rang up Soap P18
Ana rang up Rice P55
Ben rang up Coffee P9
Ana's total: P67
Ben's total: P27
Store total: P94
```

Ben starts 100 ms after Ana, so the lines alternate the same way every run.

Testing tip: before you present, run your program at least **four** ways: (1) normally; (2) **three times in a row**: the totals must be identical every time; (3) change one `sleep` time: the order of the messages changes, but the totals do not; (4) give one helper bad data, such as a negative quantity: it must be caught, and the others must still finish. No run may end in `Exception in thread "main"`, and none may hang.

Plan before you type — fill this in, and show it to your teacher:

Helper (thread)	Thread or Runnable?	Its job (what it does, how long it takes)	Its own data	Which tool joins it in, and why?

Starter template for one helper class:

```
public class ______ implements Runnable {   // or: extends Thread
    private ______ total = 0;                // ITS OWN data only
    private CountDownLatch done;

    public void run() {
        try {
            for (int i = 0; i < ______; i++) {
                // ... one piece of work ...
                Thread.sleep(______);
            }
        } catch (InterruptedException e) {
            System.out.println("______ was interrupted");
        } finally {
            done.countDown();                // always report in
        }
    }
}
// in main: new CountDownLatch(__), new Thread(job, "name").start(),
//          then done.await() inside try / catch
```

Self-check before you present — tick only what is honestly true:

I can explain, using my own code...	Yes	Not yet
why I call `start()` and not `run()`	☐	☐
what my latch (or barrier or semaphore) counts	☐	☐
why `countDown()` is in `finally`	☐	☐
why my threads do not share a variable	☐	☐

Friday: your 5-minute presentation, in this order

1. **Introduce** your theme and your helpers. *(30 sec)*
2. **Run it** and show the output. *(1 min)*
3. **Prove it live:** run it again; change a `sleep`; break it on purpose. *(2 min)*
4. **Point to the code** for a thread, the latch, and the second tool. *(1 min)*
5. **Answer questions.** *(30 sec)*

Prepare for these questions:

- Why `start()` and not `run()`?
- What does your latch count? What if the number is too big?
- Why is `countDown()` in `finally`?
- Why print the totals after the threads finish?
- What could go wrong if two threads changed one variable?

Scoring Rubric (50 points)

Criterion	Points	Score
Threads: 3+ helpers started with `start()`; named; finish before results are shown	8	
Design: `Thread` or `Runnable` used correctly; each thread keeps its own data	8	
Tools: latch used correctly; a second tool that fits the story	10	
Results: same correct totals on every run, in a fixed order	6	
Errors: `InterruptedException` and bad data handled; no crash, no hang	8	
Presentation and Q&A: proves each part live; explains choices with own code	10	
TOTAL	**50**	

LIPLET · FRIDAY After your presentation, answer the **FRIDAY strip (top of page 8)**, in pen, and hand in that sheet.

WEDNESDAY · NOV 4 **Runnable and CountDownLatch**

Name: _____
Section: _____ Score: ____ / 15

Answer after Step 5 on page 3. Tear this strip off the top of the sheet.

2.1 Define (5 pts): What is a `Runnable`? How do you get a Runnable job done (Step 4)? Give one reason it is often better than `extends Thread`.

2.2 Explain (5 pts): In Step 4, the tricycle's job stopped with an `OverloadException`, but the jeepney's job finished normally. Why did one stopped job not stop the other?

2.3 Connect (5 pts): In Step 5, `latch.countDown()` is inside `finally`. Which Week 13 idea is this? What could happen to `await()` if it were not there and a job failed?

✂ CUT ON TUESDAY · DETACH THE TUESDAY STRIP BELOW THIS LINE ✂

TUESDAY · NOV 3 **Threads: start, run, and join**

Name: _____
Section: _____ Score: ____ / 20

Answer after Step 3 on page 2.

1.1 Define (5 pts): What is a thread? How is a thread different from a process? Use the terminal analogy or your own words.

1.2 Explain (5 pts): In Step 2, the tricycle "ran on thread: main" but the bus ran on thread BUS 789. What did the code do differently?

1.3 Identify (5 pts): Write the method or state.
a) begins a new thread _____ b) waits until a thread ends _____
c) state before `start()` _____ d) state after `run()` ends _____

1.4 Connect (5 pts): `DispatchThread` says `extends Thread` and `@Override public void run()`. Which two ideas from Week 12 is that? Why must `join()` be inside `try / catch` (Week 13)?

✂ CUT ON TUESDAY · DETACH THE TUESDAY STRIP ABOVE THIS LINE ✂

PRACTICE **WEEK 16 PRACTICE BOX** — keep this part, do not hand it in. There is no Monday strip this week because there was no class on November 2.

P1. A `CountDownLatch(3)` is created. Two threads call `countDown()`. Does `await()` in `main` continue yet? Why?

P2. A `Semaphore(3)` and 5 vehicles arrive at the same moment. How many load at once? How many wait?

P3. Choose the tool: (a) wait *once* until 5 reports arrive; (b) 4 jeepneys wait for each other at every stop; (c) only 2 cashiers allowed at once.

P4. Write the one line that begins a `Thread` named `t`, and the one line that makes `main` wait for it.

Check your answers: P1 No: the count is still 1. · P2 3 load, 2 wait. · P3 (a) `CountDownLatch` (b) `CyclicBarrier` (c) `Semaphore`. · P4 `t.start();` and `t.join();` (inside try / catch).

FRIDAY · NOV 6 Performance Task Exit Questions

Name: _____
Section: _____ Score: ____ / 20

Answer after your presentation, about your own program. Hand in this whole sheet.

4.1 Point (5 pts): Name two of your helpers (threads). Write the line of your code that starts one of them, and the line that waits for it or for all of them.

4.2 Evaluate (5 pts): You used a `CountDownLatch` and a second tool (`Semaphore` or `CyclicBarrier`). Defend each choice. What would go wrong if you swapped them?

4.3 Apply (5 pts): The school canteen has 3 cashiers who serve students at the same time, and the canteen opens only when all 3 are ready. Which of this week's tools would you use for each need, and why?

4.4 Reflect (5 pts): Compare your programs before this week, where one thing happened at a time, with now. What can your program do now that it could not before? What part was hardest?

✂ CUT ON THURSDAY · DETACH THE THURSDAY STRIP BELOW THIS LINE ✂

THURSDAY · NOV 5 CyclicBarrier and Semaphore

Name: _____
Section: _____ Score: ____ / 15

Answer after Step 7 on page 4.

3.1 Define (5 pts): How is a `CyclicBarrier(3)` different from a `CountDownLatch(3)`? Say who waits and whether it can be used again.

3.2 Explain (5 pts): In Step 7, NYB 321 arrived when `Free bays` was 0. When did it start loading, and what made that possible?

3.3 Connect (5 pts): In Step 7, the four threads are kept in `List<Thread> line` and started with a `for` loop. Which earlier week taught this? Why is it handy here, and why is `release()` inside `finally`?

Week 16 CodeShare practice additions

Use this appendix with the original module. Original lesson pages and sample listings remain unchanged. Text backups and data remain unchanged in the supporting-files folder.

Alteration output avoids elapsed-time and thread-order checks. Original samples are unchanged; Steps 3, 5, 6 and 7 still contain timing/order assumptions and may not verify reliably. This JSON needs sample review before classroom rollout.

Run the unchanged sample first. Sample verified records the demonstration only. Make Your task changes, then compare the complete Expected task output. A successful alteration earns Completed. Keep module work, personal work and submitted versions separate. Paper liplets remain teacher assessed.

Step 1 alteration

Sample: java-2-w16-d2-s1 Task: java-2-w16-d2-s1-alter-v1

Run three trips instead of five. Keep start() and join(). Update the final label to "Collected after 3 trips: P".

Expected task output

```
main() runs on thread: main
Before start(): NEW
After join(): TERMINATED
NXA 123 ran on thread: NXA 123
Collected after 3 trips: P45.0
```

Step 2 alteration

Sample: java-2-w16-d2-s2 Task: java-2-w16-d2-s2-alter-v1

Correct the Tricycle worker: call start(), then join() before printing its thread name. Keep the Bus worker unchanged.

Expected task output

```
TRC 456 ran on thread: TRC 456
BUS 789 ran on thread: BUS 789
```

Step 3 alteration

Sample: java-2-w16-d2-s3 Task: java-2-w16-d2-s3-alter-v1

Run two trips for every worker instead of five. Keep starting all workers before joining. Remove the begin/ms measurements and the "Finished in under 1 second?" output; elapsed time is not a pass condition.

Expected task output

```
NXA 123: P30.0
TRC 456: P140.0
BUS 789: P55.0
Total collected: P225.0
```

Step 4 alteration

Sample: java-2-w16-d3-s4 Task: java-2-w16-d3-s4-alter-v1

Change the Tricycle groups from {2, 2, 1} to {1, 1, 1}. It should finish normally with three passengers. Keep the Jeepney groups unchanged.

Expected task output

```
Jeep loader: all boarded
Trike loader: all boarded
NXA 123 | Driver: Mang Ramon | Passengers: 19/20
TRC 456 | Driver: Kuya Ben | Passengers: 3/4
```

Step 5 alteration

Sample: java-2-w16-d3-s5 Task: java-2-w16-d3-s5-alter-v1

Use a CountDownLatch of three and remove the BUS 789 worker. Remove worker-thread ready/interruption prints from ReadyReport. After await(), print only the remaining count and final dispatch message. Remove the initial dispatcher/count print; scheduling must not decide the output.

Expected task output

```
Reports still needed: 0
All vehicles ready. Dispatch begins!
```

Step 6 alteration

Sample: java-2-w16-d4-s6 Task: java-2-w16-d4-s6-alter-v1

Use only NXA 123 and BUS 789 with a CyclicBarrier of two. Remove the NYB 321 worker and its start/join calls. Remove per-vehicle arrival prints. Put the all-here message in the barrier action so it finishes before either worker continues to the next stop. Number its two releases as "Convoy release 1" and "Convoy release 2"; keep the final Convoy finished message.

Expected task output

```
Convoy release 1
Convoy release 2
Convoy finished.
```

Step 7 alteration

Sample: java-2-w16-d4-s7 Task: java-2-w16-d4-s7-alter-v1

Use one loading bay instead of two. Remove prints from inside LoadingBay worker threads, retaining acquire/release and interruption handling. Keep all four workers and joins. After the joins print only "All loaded. Free bays: 1".

Expected task output

```
All loaded. Free bays: 1
```

Before classroom use

Upload validation is not runtime verification. Alteration output avoids elapsed-time and thread-order checks. Original samples are unchanged; Steps 3, 5, 6 and 7 still contain timing/order assumptions and may not verify reliably. This JSON needs sample review before classroom rollout.